

A. Kernel Eigenfaces

A.1 Code with Detail Explanations

Part1) Notify: **The program starts with zip file.**

Data Readin

```
1 import zipfile
2 import os
3 import numpy as np
4 import cv2
5
6 # Unzip
7 zip_path = "Yale_Face_Database.zip"
8 extract_dir = "Yale_Face_Database"
9 with zipfile.ZipFile(zip_path, 'r') as zip_ref:
10     zip_ref.extractall(extract_dir)
11
12 def load_images_from_folder(folder, img_size=(64, 64)):
13     data, labels = [], []
14     for img_file in os.listdir(folder):
15         if img_file.endswith(".pgm"): #Read in pgm
16             img_path = os.path.join(folder, img_file)
17             img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
18             img = cv2.resize(img, img_size)
19             data.append(img)
20             #labels.append(img_file.split('.')[0])
21             labels.append(int(img_file[7:9]))
22     return np.array(data), np.array(labels)
23
24 train_dir = os.path.join(extract_dir, "Yale_Face_Database", "Training")
25 test_dir = os.path.join(extract_dir, "Yale_Face_Database", "Testing")
26
27 X_train, y_train = load_images_from_folder(train_dir)
28 X_test, y_test = load_images_from_folder(test_dir)
29
30 print("Train shape:", X_train.shape)
31 print("Test shape:", X_test.shape)
```

Unzip and read in data.

PCA

```
1 def Covariance_Matrix_Computation(X):
2     N = X.shape[0]
3     d = X.shape[1] * X.shape[2]
4
5     X_flat = X.reshape(N, d)
6     Mean = np.mean(X_flat, axis=0) #64x64
7     X_centered = X_flat - Mean
8     S = np.matmul(X_centered.T, X_centered) / N
9     return S, Mean, X_flat
10 S, mu, x_flat = Covariance_Matrix_Computation(X_train)
11 print(S.shape)
12
13 eigvals, eigvecs = np.linalg.eigh(S)
14 idx = np.argsort(eigvals)[::-1]
15 eigvals = eigvals[idx]
16 eigvecs = eigvecs[:, idx]
17
18 k = 25
19 W = eigvecs[:, :k]
20
21 Z = (x_flat - mu) @ W
22 X_hat = (Z @ W.T) + mu
```

Ref: Unsupervised_Learning.pdf p.119 / p.121

Covariance Matrix Computation: $S = \frac{1}{N}(X - \mu)^T \cdot (X - \mu)$

1. Compute Covariance Matrix to find eigenvectors and eigenvalues, choose first 25 eigenvectors.

2. " xWW^T " is linear combination of principal components

which tries to reconstruct original x . $\rightarrow$ We further minus

μ for shifting distribution.

LDA

```
1 import numpy as np
2
3 def compute_scatter_matrices(Y, y_labels):
4     classes = np.unique(y_labels)
5     M, n = Y.shape
6     mu = np.mean(Y, axis=0)
7
8     S_w = np.zeros((n, n))
9     S_b = np.zeros((n, n))
10
11     for c in classes:
12         Y_c = Y[y_labels == c]
13         mu_c = np.mean(Y_c, axis=0)
14
15         #within group scatter
16         for y in Y_c:
17             diff = (y - mu_c).reshape(-1, 1)
18             S_w += diff @ diff.T
19
20         #between group scatter
21         n_c = Y_c.shape[0]
22         diff_mu = (mu_c - mu).reshape(-1, 1)
23         S_b += n_c * (diff_mu @ diff_mu.T)
24
25     return S_w, S_b
```

Compute Scatter Matrices contains S_w and S_b .

Within Group: Samples with their own cluster center.

$$S_w = \sum_{c=1}^C \sum_{x_i \in c} (x_i - \mu_c)(x_i - \mu_c)^T$$

Between Groups: Cluster centers with Total center.

$$S_b = \sum (\mu - \mu_c)(\mu - \mu_c)^T$$

```

27 def lda_eigen(S_w, S_b, num_components):
28
29     S_w_reg = S_w + 1e-6 * np.eye(S_w.shape[0]) # Avoid singular
30
31     eigvals, eigvecs = np.linalg.eig(np.linalg.inv(S_w_reg) @ S_b)
32
33     # Sort
34     idx = np.argsort(eigvals)[::-1]
35     eigvecs = eigvecs[:, idx]
36
37     # First num_components eigenvectors
38     U = eigvecs[:, :num_components]
39     return U.real
40
41 def compute_fisherfaces(W_pca, U):
42     W_lda = W_pca @ U
43     return W_lda
44
45 X_centered = x_flat - mu
46 Y_train = X_centered @ W
47
48 S_w, S_b = compute_scatter_matrices(Y_train, y_train)
49
50 num_classes = len(np.unique(y_train))
51 U = lda_eigen(S_w, S_b, num_classes - 1)
52
53 W_lda = compute_fisherfaces(W, U)

```

Lda Eigen: (Ref: Unsupervised_Learning.pdf p.176)

From Rayleigh quotient: $S_B W = \lambda S_W W \rightarrow \text{Get } \lambda \cdot (U)$

Compute Fisherfaces: Fisherfaces can be calculate from PCA result.

1. Projection: $z = Wx$. (Y_trained here)

2. Fisherfaces can only built with Num_classes-1 eigenvectors.

Image Rebuilt

```

1 import random
2 import matplotlib.pyplot as plt
3
4 nums = random.sample(range(0, y_train.shape[0]), 10)
5 # PCA rebuilt
6 for i in nums:
7     x_i = x_flat[i]
8     x_pca = W.T @ (x_i - mu)
9     x_hat_pca = mu + W @ x_pca
10    x_hat_pca = x_hat_pca.reshape(64, 64)
11    # reshape back to 64 x 64
12    plt.imshow(x_hat_pca, cmap='gray')
13    plt.title('PCA REBUILT, Pic Index: %d'%i)
14    plt.show()
15
16 # LDA rebuilt
17 for i in nums:
18     x_i = x_flat[i]
19     x_lda = W_lda.T @ (x_i - mu)
20     x_hat_lda = mu + W_lda @ x_lda
21     x_hat_lda = x_hat_lda.reshape(64, 64)
22     # reshape back to 64 x 64
23     plt.imshow(x_hat_lda, cmap='gray')
24     plt.title('LDA REBUILT, Pic Index: %d'%i)
25     plt.show()

```

For both PCA and LDA:

1. Rebuilt: Projection to spaces then project it back.

2. $z = Wx \rightarrow z \cdot W^T$

Part2)

Part2

```

1 from scipy.spatial.distance import cdist
2 from collections import Counter
3
4 def Z_test_computation(X_test_flat, mu, W_proj):
5     X_test_centered = X_test_flat - mu
6     Z_test = X_test_centered @ W_proj
7     return Z_test
8
9 def Clustering_and_Results(Z_test, Z_train, y_train, y_test, k=3):
10    D = cdist(Z_test, Z_train, metric='euclidean') # shape: (N_test, N_train)
11    pred_labels = []
12
13    for i in range(D.shape[0]):
14        knn_idx = np.argsort(D[i])[0:k]
15        knn_labels = y_train[knn_idx]
16        pred = Counter(knn_labels).most_common(1)[0][0]
17        pred_labels.append(pred)
18
19    # Accuracy calculation
20    correct = sum(pred_labels[i] == y_test[i] for i in range(len(y_test)))
21    acc = correct / len(y_test)
22    print(f'Accuracy (k={k}): {acc:.4f}')
23    return pred_labels
24
25 x_test_flat = X_test.reshape(-1, 4096)
26 # PCA Projection
27 Z_train_pca = (x_flat - mu) @ W
28 Z_test_pca = Z_test_computation(x_test_flat, mu, W)
29 PCA_Test_Pred = Clustering_and_Results(Z_test_pca, Z_train_pca, y_train, y_test, k=3)
30
31 # LDA Projection
32 Z_train_lda = (x_flat - mu) @ W_lda
33 Z_test_lda = Z_test_computation(x_test_flat, mu, W_lda)
34 LDA_Test_Pred = Clustering_and_Results(Z_test_lda, Z_train_lda, y_train, y_test, k=3)

```

Test Accuracy with test dataset.

1. Concept: Calculate distance between test and train (how similar they are). $\rightarrow$ Built D.

2. Find most similar result with k-nn.

3. Statistic for all samples.

Part3)

Part3

```

1 def linear_kernel(X1, X2):
2     return X1 @ X2.T
3
4 def RBF_kernel(X1, X2, sigma=0.01):
5     diff = X1[:, None] - X2[None, :]
6     return np.exp(-np.linalg.norm(diff, axis=-1)**2 / (2 * sigma**2))
7
8 # Calculate centered data of train and test
9 mean_face = x_flat.mean(axis=0)
10
11 x_flat_c = x_flat - mean_face
12 x_test_flat_c = x_test_flat - mean_face

```

Kernel Methods

1. Linear Kernel: $LK(x, x') = x^T x'$

2. RBF Kernel: $RK(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$

3. Calculate centered data for later use.

Linear Kernel PCA

```

1 #Gram Matrix
2 K = linear_kernel(x_flat_c, x_flat_c)
3
4 #Eigen
5 eigvals, eigvecs = np.linalg.eigh(K)
6 idx = np.argsort(eigvals)[::-1]
7
8 eigvals = eigvals[idx]
9 eigvecs = eigvecs[:, idx]
10
11 alphas = eigvecs[:, :k] / np.sqrt(eigvals[:k] + 1e-10)
12
13 #Projection
14 Z_train = K @ alphas
15
16
17 #cross-kernel
18 K_test = linear_kernel(x_test_flat_c, x_flat_c)
19
20 Z_test = K_test @ alphas
21
22 #Result
23 LK_PCA = Clustering_and_Results(Z_test, Z_train, y_train, y_test, k=0)

```

```

3 def centralize_kernel(K):
4     N = K.shape[0]
5     one = np.ones((N, N)) / N
6     return K - one @ K - K @ one + one @ K @ one
7
8 def centralize_cross_kernel(K_train, K_test):
9     train_col_mean = K_train.mean(axis=0)
10    train_mean = K_train.mean()
11    test_row_mean = K_test.mean(axis=1)
12    return K_test - test_row_mean[:, None] - train_col_mean[None, :] + train_mean
13
14 def kernel_lda_linear(K_train_c, y, reg=1e-3):
15     N = K_train_c.shape[0]
16     classes = np.unique(y)
17
18     mean_global = K_train_c.mean(axis=0)
19
20     Sw = np.zeros((N, N))
21     Sb = np.zeros((N, N))
22
23     for c in classes:
24         idx = np.where(y == c)[0]
25         Kc = K_train_c[idx, :]
26         mean_c = Kc.mean(axis=0)
27
28         # within-class scatter
29         diff = Kc - mean_c
30         Sw += diff.T @ diff
31
32         # between-class scatter
33         nd = (mean_c - mean_global).reshape(-1, 1)
34         Sb += len(idx) * (nd @ nd.T)
35
36     Sw += reg * np.eye(N)
37     eigvals, eigvecs = np.linalg.eigh(np.linalg.inv(Sw) @ Sb)
38     order = np.argsort(eigvals)[::-1]
39     W = eigvecs[:, order][:, :len(classes) - 1]
40     return W

```

```

42 def project_train_test(K_train_c, K_test_c, W):
43     Z_train = K_train_c @ W
44     Z_test = K_test_c @ W
45     return Z_train, Z_test
46
47 K_train = linear_kernel(x_flat, x_flat)
48 K_train_c = centralize_kernel(K_train)
49
50 K_test = linear_kernel(x_test_flat, x_flat)
51 K_test_c = centralize_cross_kernel(K_train, K_test)
52
53 W = kernel_lda_linear(K_train_c, y_train, reg=1e-6)
54
55 Z_train, Z_test = project_train_test(K_train_c, K_test_c, W)
56
57 LK_LDA = Clustering_and_Results(Z_test, Z_train, y_train, y_test, k=0)

```

RBF Kernel PCA

```

1 sigma=10000
2 K = RBF_kernel(x_flat_c, x_flat_c, sigma=sigma)
3
4 eigvals, eigvecs = np.linalg.eigh(K)
5 idx = np.argsort(eigvals)[::-1]
6
7 eigvals = eigvals[idx]
8 eigvecs = eigvecs[:, idx]
9
10 alphas = eigvecs[:, :k] / np.sqrt(eigvals[:k] + 1e-10)
11
12 Z_train = K @ alphas
13
14 K_test = RBF_kernel(x_test_flat_c, x_flat_c, sigma=sigma)
15
16 Z_test = K_test @ alphas
17
18 RBF_PCA = Clustering_and_Results(Z_test, Z_train, y_train, y_test, k=0)

```

Accuracy (k=0): 0.8333

RBF Kernel LDA

```

1 sigma=10000
2 K_train = RBF_kernel(x_flat, x_flat, sigma=sigma)
3 K_train_c = centralize_kernel(K_train)
4
5 K_test = RBF_kernel(x_test_flat, x_flat, sigma=sigma)
6 K_test_c = centralize_cross_kernel(K_train, K_test)
7
8 W = kernel_lda_linear(K_train_c, y_train, reg=1e-3)
9
10 Z_train, Z_test = project_train_test(K_train_c, K_test_c, W)
11
12 RBF_LDA = Clustering_and_Results(Z_test, Z_train, y_train, y_test, k=0)

```

Accuracy (k=0): 0.7000

Linear Kernel PCA:

1. Concept: Replaced Covariance Matrix with Gram Matrix.
2. Projection formula: $Z = K\alpha$
3. The other part is same as before: Calculate for test and train, clustering and results.

→ In this section, I choose re-calculate instead of using variables from PCA.

Centralize Kernel (Ref: Unsupervised_Learning.pdf p.128)

$K^c = K - 1_N K - K 1_N + 1_N K 1_N$, In kernel PCA, we can't directly centralize it, since feature space may has infinite dimensions.

Centralize Cross Kernel

This procedure ensures alignment of projection between train data and test data.

Linear Kernel LDA:

Calculate same thing as Part1's LDA, but with kernel-based.

Project train test: (This W is different from part1, 2's)

$$z = \sum_i \alpha_i K(x_i, x_{new}) \rightarrow Z = KW$$

1. Centralize Kernel after each kernel computation.
2. Other steps are similar as part1,2.

RBF Kernel PCA:

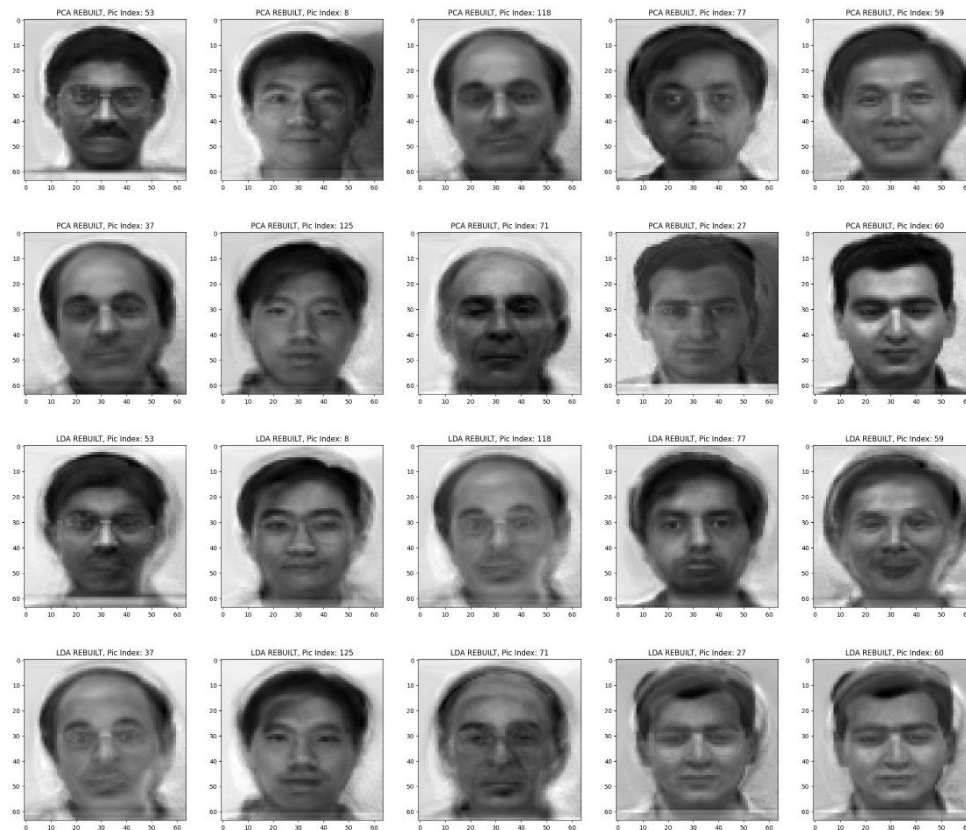
1. For better performance, I set sigma=10000 for both PCA and LDA.
2. The only change here is kernel: Replaced linear kernel computation with RBF kernel.

RBF Kernel LDA:

1. Same here, change LDA calculation from linear based to kernel based.

A.2 Experiments and Discussion

Part1) Randomly Pick 10 pictures for construction. (For PCA / LDA)



Part2) PCA Test Accuracy (k=3): 0.8333 / LDA Test Accuracy (k=3): 0.9667

Part3)

1. Linear Kernel PCA Test Accuracy (k=3): 0.8333
2. Linear Kernel LDA Test Accuracy (k=3): 0.1000
3. RBF Kernel PCA Test Accuracy (k=3, σ in RBF=10000): 0.8333
4. RBF Kernel LDA Test Accuracy (k=3, σ in RBF=10000): 0.7000

Discussion: In Part2, we found LDA has better performance than PCA, the performance can be treated as reference for later comparison. → Kernel methods should have similar performance.

However, in part3, LDA's performance is pretty strange.

And for RBF Kernel LDA, setting σ differently strongly affects performance since k-NN may easily lead to the same cluster. For example, if $\sigma = 80000$, accuracy = 0.8333; $\sigma = 2000$, accuracy = 0.1667.

B. t-SNE

B.1 Code with Detail Explanations

Part1) (Actually Part2, Part3 are included in this part.) (Look out for file extraction path)

→I only change **tsne** function in tsne.py:

```
107 def tsne(X=np.array([]), switch='tsne', no_dims=2, initial_dims=50, perplexity=30.0):
108     """
109     Runs t-SNE on the dataset in the NxD array X to reduce its
110     dimensionality to no_dims dimensions. The syntax of the function is
111     'Y = tsne.tsne(X, no_dims, perplexity)', where X is an NxD NumPy array.
112     """
113
114     # Check inputs
115     if isinstance(no_dims, float):
116         print("Error: array X should have type float.")
117         return -1
118     if round(no_dims) != no_dims:
119         print("Error: number of dimensions should be an integer.")
120         return -1
121
122     # Initialize variables
123     X = pca(X, initial_dims).real
124     (n, d) = X.shape
125     max_iter = 200
126     initial_momentum = 0.5
127     final_momentum = 0.8
128     eta = 500
129     min_gain = 0.01
130     Y = np.random.randn(n, no_dims)
131     dY = np.zeros((n, no_dims))
132     iY = np.zeros((n, no_dims))
133     gains = np.ones((n, no_dims))
134
135     # Compute P-values
136     P = x2p(X, le=5, perplexity)
137     P = P + np.transpose(P)
138     P = P / np.sum(P)
139     P = P * 4. # early exaggeration
140     P = np.maximum(P, 1e-12)
141
142     # Run iterations
143     Frames = []
144     for iter in range(max_iter):
145
146         # Compute pairwise affinities
147         sum_Y = np.sum(np.square(Y), 1)
148         num = -2. * np.dot(Y, Y.T)
149         if switch == 'tsne':
150             num = 1. / (1. + np.add(np.add(num, sum_Y).T, sum_Y))
151         elif switch == 'symmetric':
152             num = np.exp(-np.add(np.add(num, sum_Y).T, sum_Y))
153
154         num[num>0, num>0] = 0.
155         Q = num / np.sum(num)
156         Q = np.maximum(Q, 1e-12)
157
158         # Compute gradient
159         PQ = P - Q
160         for i in range(n):
161             dY[i, :] = np.sum(np.tile(PQ[:, i] * num[:, i], (no_dims, 1)).T * (Y[i, :] - Y), 0)
162
163         # Perform the update
164         if iter < 20:
165             momentum = initial_momentum
166         else:
167             momentum = final_momentum
168         gains = (gains + 0.2) * ((dY > 0.) != (iY > 0.)) + \
169             (gains * 0.8) * ((dY > 0.) == (iY > 0.))
170         gains[gains < min_gain] = min_gain
171         iY = momentum * iY - eta * (gains * dY)
172         Y = Y + iY
173         Y = Y - np.tile(np.mean(Y, 0), (n, 1))
174
175         # Compute current value of cost function
176         if (iter + 1) % 10 == 0:
177             C = np.sum(P * np.log(P / Q))
178             print("Iteration %d: error is %f" % (iter + 1, C))
179             Frames.append(Y.copy())
180
181         # Stop lying about P-values
182         if iter == 100:
183             P = P / 4.
184
185     # Return solution
186     return Y, Frames, P, Q
```

```
5 def scatter_to_image(Y, labels, input_k, cmap='tab10'):
6     plt.figure(figsize=(6,6))
7     # 使用 matplotlib colormap
8     colors = plt.cm.get_cmap(cmap, input_k)(labels.astype(int))
9     plt.scatter(Y[:,0], Y[:,1], c=colors, s=5)
10    plt.axis('off')
11    plt.tight_layout()
12
13    # 生成暫存檔再讀回 numpy array
14    plt.savefig('temp.png')
15    plt.close()
16    img = imageio.imread("temp.png")
17    return img
18
19 def save_embedding_gif(frames, labels, input_k, filename):
20    gif_frames = [scatter_to_image(f, labels, input_k) for f in frames]
21    imageio.mimsave(filename+'.gif', gif_frames, duration=0.5)
22
23 def visualize_similarity_distribution(P, Q, title, LOG=False):
24    plt.figure(figsize=(10,4))
25    plt.subplot(1,2,1)
26    plt.hist(P.flatten(), bins=100, alpha=0.7)
27    if LOG:
28        plt.yscale('log')
29    plt.title("High-dimensional P distribution, %s"%title)
30    plt.subplot(1,2,2)
31    plt.hist(Q.flatten(), bins=100, alpha=0.7)
32    if LOG:
33        plt.yscale('log')
34    plt.title("Low-dimensional Q distribution, %s"%title)
35    plt.show()
```

(Ref: Unsupervised_Learning.pdf p.166 / p.172)

Only difference between t-SNE and Symmetric SNE: q_{ij} .

$$\text{t-SNE: } q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$

$$\text{Symmetric SNE: } q_{ij} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq l} \exp(-\|y_l - y_k\|^2)}$$

→Change into switch-mode: Line149 – 152.

tsne (modified)

1. Calculate similarities matrix P, Q.

(P: High-dimensional ; Q: Low-dimensional)

2. Calculate KL divergence to see how close these two distribution are.

$$C = KL(p||q) = \sum_{i \neq j} P_{ij} \cdot \log \frac{P_{ij}}{Q_{ij}}$$

3. Iteration to minimize C. → $\frac{dC}{dY}$

$$\frac{dC}{dY} = 4 \sum_j (P_{ij} - Q_{ij})(y_i - y_j)(1 + \|y_i - y_j\|^2)^{-1}$$

Decrease gradient to adjust Y. → Q and P get closer.

Scatter to image: Draw data points in 2D, with different color for each cluster.

Save embedding gif: Concatenate input frames.

Visualize similarity distribution: (used in part2)

1. Plot P and Q side by side.

2. Since distributions' different were more obvious in log scale, I take y scale with log.

```

37 def part123_main(sne_type):
38     print("Run Y = tsne.tsne(X, no_dims, perplexity) to perform t-SNE on your dataset.")
39     print("Running example on 2,500 MNIST digits...")
40     X = np.loadtxt("tsne_python/tsne_python/mnist2500_X.txt")
41     labels = np.loadtxt("tsne_python/tsne_python/mnist2500_labels.txt")
42     Y, frames, P, Q = tsne(X, sne_type, 2, 50, 20.0)
43     save_embedding_gif(frames, labels, input_k=10, filename=sne_type+"_embedding")
44     pylab.scatter(Y[:,0], Y[:,1], c=labels, cmap='tab10', s=20)
45     pylab.show()
46     visualize_similarity_distribution(np.array(P), np.array(Q), sne_type, LOG=True)
47
48 part123_main('tsne')
49 part123_main('symmetric')

```

Part123 main: (Perplexity = 20.0)

1. Load in data → Do tsne and Symmetric SNE. (Part1)
2. Save embedding result to gif and plot scatter graph. (Part2)
3. Plot P, Q distributions (Part3)

Part2) Show projection and optimize procedure (Integrated in Part123 main)

Part3) Visualize the distribution of pairwise similarities. (Integrated in Part123 main)

Part4) Change Perplexity: [10, 30] (Since we have 20 before.)

```

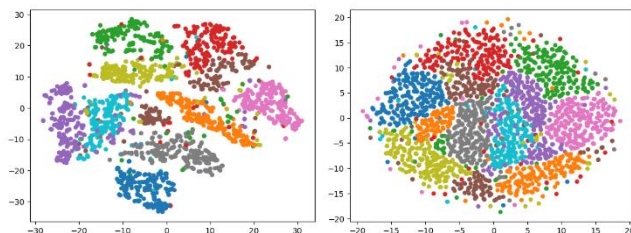
1 def part4_main(sne_type):
2     print("Run Y = tsne.tsne(X, no_dims, perplexity) to perform t-SNE on your dataset.")
3     print("Running example on 2,500 MNIST digits...")
4     X = np.loadtxt("tsne_python/tsne_python/mnist2500_X.txt")
5     labels = np.loadtxt("tsne_python/tsne_python/mnist2500_labels.txt")
6
7     Perplexity_Setting = [10, 30] #20前面試過了
8     for per in Perplexity_Setting:
9         Y, frames, P, Q = tsne(X, sne_type, 2, 50, per)
10        save_embedding_gif(frames, labels, input_k=10, filename=sne_type+"_embedding_P_"+str(per))
11        pylab.scatter(Y[:,0], Y[:,1], c=labels, cmap='tab10', s=20)
12        pylab.show()
13        visualize_similarity_distribution(np.array(P), np.array(Q), sne_type, LOG=True)
14
15 part4_main('tsne')
16 part4_main('symmetric')

```

Choose different perplexity for each iteration, both 'tsne' and 'symmetric SNE'.

B.2 Experiments and Discussion

Part1) Clustering Results (Left: t-SNE, Right: Symmetric SNE) (Perplexity = 20)



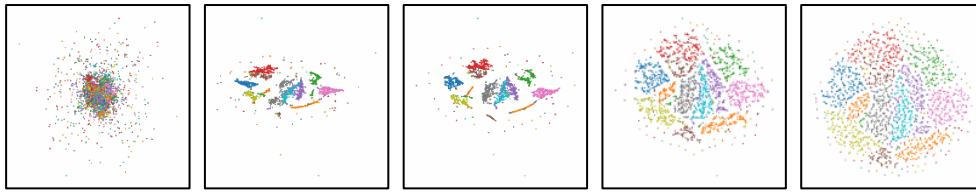
Observation: crowded problem, symmetric SNE failed to effectively separate the clusters from each other in such lower dimension. (Project 10 classes into 2D.)

Part2) Show 2D distributions and GIF frames. → 2D distributions shown in Part1.

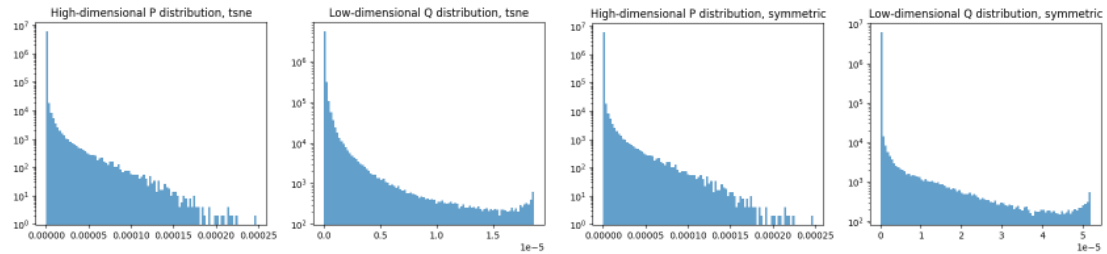
GIF frames of t-SNE (Iteration: [0, 40, 80, 120, 160], Total Iteration: 200)



GIF frames of Symmetric-SNE (Iteration: [0, 40, 80, 120, 160], Total Iteration: 200)



Part3) Visualize the distribution of pairwise similarities (Y in log scale)

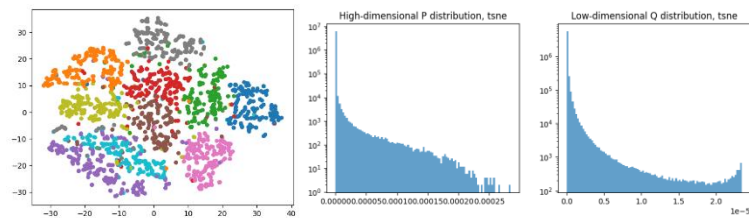


Shows: tSNE strongly squeeze “non-neighbors” → Q is closer to 0.

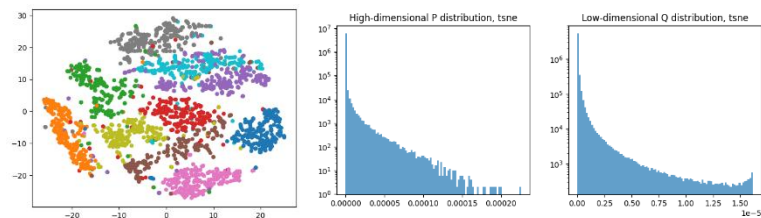
Symmetric SNE has smoother distribution. → Clusters may be “unclear”.

Part4) Different Perplexity Setting

Perplexity = 10, tSNE.



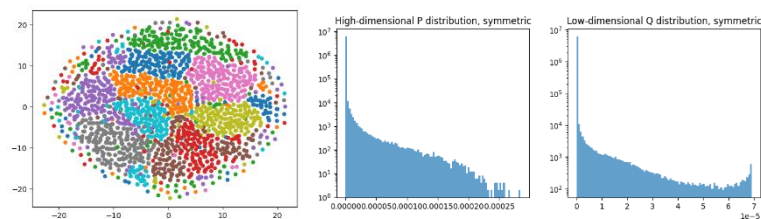
Perplexity = 30, tSNE.



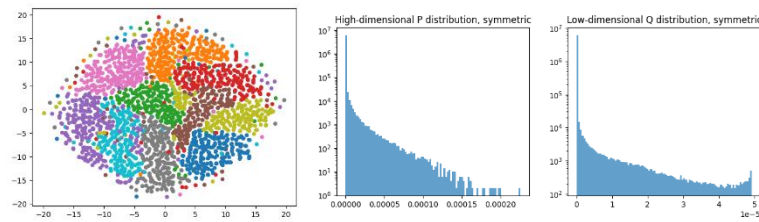
As perplexity increase from 10 to 30:

1. Q distribution's tail has been shorter.
2. P distribution also sharper.

Perplexity = 10, Symmetric SNE.



Perplexity = 30, Symmetric SNE.



As perplexity increase from 10 to 30:

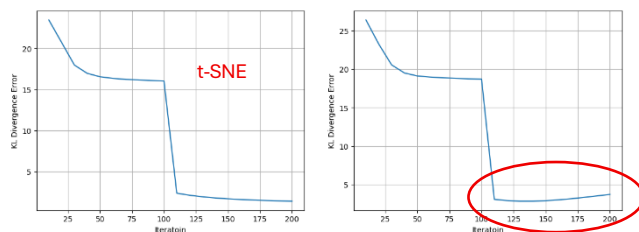
1. Q distribution's tail has been shorter.
2. P distribution also sharper.

(As same as tSNE)

B.3 Observations and Discussion

For Quest1: Different sigma setting in RBF kernel change test performance. Also, it's strange that linear kernel LDA has such bad performance. (In meanwhile, linear kernel PCA and RBF kernel LDA works normally.) (For Detail, see "Discussion" after A.2 part3)

For Quest2: We found error "bounce back" in symmetric SNE (right).



Code Notify:

*First Quest → Read in zip file directly. (See A.1)

*Second Quest → Unzip first, then read data inside only. (Modified t-SNE already contains in first section, See B.1)